

GameMakerKit bootstrap — davila7 agents + UnrealClaude

Adoption analysis, 2026-05-27. For Max Schenk's review. Companion to the agent-readable MD.

What this document is

You asked for an analysis of two upstream dependencies the GameMakerKit was opened around: davila7's five game-development subagents, and Natfii's UnrealClaude UE5 plugin. The kit-side scaffolding is in place (`ClaudeKit/GameMakerKit/`), the davila7 agents are vendored verbatim with SHA-pinned provenance, and UnrealClaude's MCP tool catalog is extracted to a reference doc. Nothing is deployed yet — everything sits in `_candidates/` or `references/` waiting on your call.

The structure below trades the agent-MD's tables for a longer reasoning trail. The recommendations are the same; the prose explains what changes if you take each one.

Section 1 — The davila7 game-dev agent set

There are five files. Three of them are interesting; two are not. The cut shakes out roughly along a "do they fill a real gap in your current kit?" axis — and the answer is mostly no, because your kit is already strong on code-side agents and weak on game-specialty agents.

`unreal-engine-developer` — strong adopt candidate

This is the headline agent in the set. The frontmatter is honest: Read, Write, Edit, Bash, and a description that frames it as a UE5 C++ + Blueprint specialist with a 10-year AAA framing. The system prompt is competent. It doesn't claim Blueprint primacy or C++ primacy — it splits the work correctly between them.

What it gives you that you don't have: UE5 idiom awareness. Your current kit has `cpp-pro` (engine-agnostic), the generic `game-developer`, and `code-reviewer`. None of those will catch a missing `UPROPERTY`, a UObject raw-pointer leak, an ungarded blocking call on the game thread, or a replication-rule violation. `unreal-engine-developer` is built around exactly those patterns.

The system prompt has two small concerns. First, it includes "Use PROACTIVELY" in its description, which combined with the main session's default proactive-use reflex could surface this agent more often than you want. If you adopt, soften that phrasing during the deploy step. Second, the system prompt is a generalist AAA voice — no Lyra, no Gameplay Ability System awareness, nothing specific to your house style. That's appropriate for an upstream template, but it means the agent's actual usefulness on TOG depends on what you encode in the project-level CLAUDE.md. The agent is good plumbing; your project file makes it sharp.

Recommendation: adopt with light editing. Vendor as-is; tweak the proactive language at deploy time; rely on project CLAUDE.md for house-style specificity.

game-designer — strong adopt candidate, lower stakes

A 40-line agent focused on mechanics, balance, progression, narrative integration, monetization frameworks. The tools field correctly omits Bash — design work doesn't execute code, so there's nothing to be careful about on the security side.

This one fills the largest gap in your current kit by a wide margin. You have zero design-specialty agents. Everything in the kit today is code, research, critique, or writing. For a combat designer shipping TOG, having a balance-and-mechanics agent on call is real complement, not theater.

Two soft caveats. The system prompt covers F2P monetization and mobile economy patterns that don't apply to TOG or TBH. That's noise, not error — the agent will still do useful work for you, it just carries some baggage. And the agent has no built-in instrumentation for validating its own balance claims (it can't see playtest data), so its outputs are theoretical until you wire them to something measurable. That's true of any design agent, but worth naming.

Recommendation: adopt. Lower deployment risk than `unreal-engine-developer` because there's no code-execution path. If `unreal-engine-developer` is the load-bearing pick, this is the easy win that costs almost nothing.

3d-artist — defer, not skip

Engine-agnostic 3D asset pipeline agent: modeling, texturing, PBR, LOD, technical art docs. Has Bash for tooling integration (Blender CLI, FBX SDK, that sort of thing).

The "do you have this work today?" answer is no. Your combat-designer scope doesn't currently include in-house 3D pipeline ownership. So adopting this agent today would put a tool in the drawer that you don't currently use. There's no harm in that, but there's also no point.

Recommendation: leave it in `_candidates/`. Revisit if your scope shifts toward tech-art work, if outsource-review becomes recurring, or if you start owning an asset-pipeline tool.

unity-game-developer — park

A competent Unity C# specialist (Unity 2022.3+, mobile, console, PC, ECS, save/load, AI). If TBH wakes up, you want this agent.

TBH is parked, so this agent is parked too. Same `_candidates/` folder, same logic — keep it ready, don't deploy.

Recommendation: park. No further action this pass.

game-developer — skip

This is the only file in the set you should actively decline to deploy.

Three reasons. First, it overlaps with your existing generic `game-developer.md` in the kit, so importing this one would create a name collision and force a rename. Second, if you adopt `unreal-engine-developer` (recommended), this agent's UE-relevant coverage is strictly worse — it's cross-

platform-generic and has no engine-specific depth. Third, and most importantly, the system prompt is built around an **orchestrator pattern** that explicitly names eight subagent collaborators (frontend-developer, backend-developer, performance-engineer, mobile-developer, devops-engineer, qa-expert, product-manager, ux-designer). Spinning this up would either cascade into work your kit doesn't have, or quietly degrade into a regular agent that pretends those collaborators exist. Neither is what you want.

This is also the broadest tool grant in the set — Read, Write, Edit, Bash, Glob, Grep. Not a problem in isolation, but combined with the orchestrator framing it's a "wants to do everything" shape that doesn't match how your kit actually works.

Recommendation: drop. Keep the vendored file in `_candidates/` purely as a vendoring record (so re-evaluating later is one read away) but don't promote it to a deploy path.

Section 1 summary

Adopt two: `unreal-engine-developer` (with the proactive-language tweak) and `game-designer` (straight). Defer `3d-artist`. Park `unity-game-developer`. Drop `game-developer`. Five agents in, two agents out.

Section 2 — UnrealClaude

UnrealClaude is a different kind of dependency from the davila7 agents. The davila7 stuff is just prompts — text files you copy into `~/.claude/agents/` and forget about. UnrealClaude is a UE5 plugin with a C++ source layer, a bundled Node.js MCP bridge, and a runtime server that lives on `localhost:3000` whenever the editor is open. It changes what Claude Code can physically do inside Unreal. That's a much bigger swing.

What it actually is

The repo ships four things that matter: the `.uplugin` manifest, the C++ plugin source under `Source/UnrealClaude/`, the `Resources/mcp-bridge/` Node submodule (which pulls `Natfii/ue5-mcp-bridge` separately — that's where the actual MCP tool surface is implemented), and a `CLAUDE.md.default` template at `UnrealClaude/CLAUDE.md.default` (note: one directory deep, not at repo root — the earlier exploration assumed it was at root).

What it doesn't ship: prebuilt binaries. You compile per platform. The maintainer has verified Win64, Linux, and macOS Apple Silicon, so the toolchain works, but plan for a one-time build cost.

How it runs

Open the editor with the plugin installed; the plugin auto-starts an HTTP MCP server on port 3000 and registers an in-editor chat widget under `Tools → Claude Assistant`. You can use the in-editor widget directly, or you can keep using Claude Code from the terminal and point it at the MCP server via `/mcp` — same tool surface either way.

The tool layout uses two routing patterns. Direct calls for read-only ops and simple writes (you call `unreal_status`, `unreal_asset_search`, `unreal_spawn_actor` directly). Domain-routed calls for complex stateful ops — you go through `unreal_ue` with a `domain:` parameter (`blueprint`, `material`, `character`, `anim`, `enhanced-input`). Concurrency is capped at four MCP tasks at once; subagent fan-out is capped at three (the maintainer reserves a slot for the lead). Sequential-only ops like `open_level`, `delete`, and `execute_script` block the queue.

Full tool catalog with safety classes is in `references/unrealclaude-mcp-tools.md` — twenty-nine rows, including the read-only set, the per-object-safe writes, and the sequential-only ops.

What it changes

The honest answer: this plugin moves Claude from "reads your repo and runs your build" to "reads your level, your asset graph, your Blueprint structure, and can execute editor Python on demand." That's a real capability shift.

Concrete new capabilities, listed roughly in order of how often you'd actually use them:

- **Asset graph queries.** `asset_dependencies` and `asset_referencers` give Claude an impact-analysis surface that doesn't exist without the plugin. For TOG-scale projects this is the single biggest day-to-day win.
- **Level introspection.** `get_level_actors` plus `spawn_actor` means Claude can survey what's in a level and place things programmatically. Useful for templating, less useful for tuning.
- **Blueprint structural reads.** `blueprint_query` gives Claude visibility into Blueprint structure (variables, functions, graphs) without parsing serialized binary data.
- **Material and VFX iteration.** Querying material parameters and modifying material instances through `unreal_ue` with `domain: material` cuts editor-GUI clicking for parameter sweeps.
- **Editor Python automation.** `execute_script` is the big one strategically — this is the surface you specifically said you wanted to lean into.

The capabilities Claude **still won't have** even with UnrealClaude installed: editing C++ source outside Live Coding scope, detailed node-level Blueprint graph wiring (the maintainer's own CLAUDE.md flags Blueprint mutation as having known bugs — quote: "don't rely on fully"), direct UE Reflection API queries, and multi-instance network testing.

How it fits your stated strategy

Your standing UE strategy, from PLAN.md and confirmed in supabrain, is to lean into Unreal's three AI-friendly surfaces — C++, Python editor scripting, and MCP — while keeping Blueprints as the human-driven layer. UnrealClaude maps to that strategy almost perfectly.

The Python-editor-scripting surface is exposed directly by `execute_script`. The MCP surface is the whole plugin. The C++ surface gets read coverage (asset/level/source-tree queries) but limited write coverage (Live Coding only). The Blueprint surface stays bounded — Claude can read Blueprints freely but its write capabilities are limited *and* acknowledged buggy, which paradoxically aligns with your "keep Blueprints human-driven" stance rather than undermining it.

The match is strong enough that, if your strategy holds, UnrealClaude isn't really optional — it's the plumbing that makes the strategy real.

Risks worth flagging

Five things to know going in.

Blueprint mutation is buggy by maintainer's admission. Use the read tools freely; use the Blueprint-write tools sparingly until the upstream issue list clears.

Large-project context injection is slow. The plugin streams modules, plugins, assets, and project settings to Claude at session start, and on TOG-scale projects this adds latency. Not a blocker; just expect a slower first response.

OneDrive and Dropbox sync hang the MCP at 60 seconds. There's a watchdog that detects it, but if you're syncing the project root through one of those services, that's a real failure mode. Per supabrain you're not on OneDrive for these projects, so this is a low-probability hit for you specifically.

The build pipeline requires `npm install` for the MCP bridge after clone, plus the standard UE plugin build. That's a one-time cost to document in the `/gamemaker-onboard` flow but not a recurring tax.

The maintainer is a single-developer indie effort. Bus-factor is one. There's one open blocker issue ("Claude cannot read Blueprints or use other custom tools") that needs status-checked before any deploy. The mitigation is the SHA-pinning already in place: GameMakerKit's reference doc pins the exact plugin commit and bridge commit, so a project can lock to a known-good combination.

Recommendation

Adopt as a documented optional dependency. Don't bundle the plugin into GameMakerKit itself — it's a UE plugin, not a Claude artifact, and a Claude kit shouldn't ship engine binaries. What GameMakerKit ships is the *integration knowledge*: this analysis, the MCP tool catalog, the `/gamemaker-onboard` check that detects whether `Plugins/UnrealClaude/` exists in a target project, and a short rider on `unreal-engine-developer`'s system prompt telling it to prefer `unreal_*` MCP tools over filesystem reads when the plugin is present.

That keeps GameMakerKit useful both in projects that have UnrealClaude installed and in projects that don't. The agent works either way; the plugin makes it sharper.

Section 3 — What `/gamemaker-onboard` will check

This is forward-looking. The full command spec is a stub this pass — full implementation lands after you confirm the agent set. Worth sketching here so the analysis hangs together with the kit's design direction.

When you run `/gamemaker-onboard` in a UE project, it should walk five checks. Read the `.uproject` to confirm UE5 and parse the engine version (warn if not 5.7, since that gates UnrealClaude). Look for `Plugins/UnrealClaude/UnrealClaude.uplugin` and offer the clone-and-build flow if absent. Check for an existing `CLAUDE.md` at the project root and either merge or offer to

drop `CLAUDE.md.gamemaker.template` with project-specific placeholders pre-filled. Show the GameMakerKit agent allowlist and let you accept or decline the default set. If UnrealClaude is installed and the editor is running, hit `http://localhost:3000/mcp/status` to confirm the bridge is alive.

What it will explicitly not do: auto-modify the `.uproject`, install UnrealClaude without per-step confirmation, or write anywhere outside `<project_root>/CLAUDE.md` and `<project_root>/claude/`. That keeps the blast radius bounded.

Section 4 — Two open questions you raised

You flagged two things alongside the main analysis request. Both are noted but deferred to separate passes, with reasoning below.

Game-dev-flavored skills — you mentioned that "skills to make sure the code review specialises in game dev behavior might be necessary." Agreed in principle. The natural candidates: a `ue5-anti-patterns` skill the existing `code-reviewer` can consume (raw UObject pointers, missing `UPROPERTY`, blocking the game thread, replication misuse); a `gameplay-ability-system-patterns` skill conditional on whether TOG uses GAS; a `combat-system-conventions` skill encoding the TOG posture/deflect/stance idioms; an `unreal-network-replication-rules` skill for replicated `UPROPERTY` and `RPC` categories. None of these are built this pass — they're skill-design work that wants its own pass after the `davila7` agent set is confirmed.

featureDev subagent — you asked whether to bring this back, "as long as it's given a skill to push it out of default behavior." The relevant context: you disabled the `feature-dev` plugin during the 2026-05-25 cleanup and the decision held. Re-importing the subagent alone (without the plugin's command/hook surface) would require fetching the upstream source, auditing it with `subagent-design-reviewer`, and writing the constraining skill you alluded to. That's a meaningful chunk of work that shouldn't get bundled into this bootstrap. Defer to its own pass, prioritize after Section 4's skill list.

Section 5 — What you actually decide from here

Four decisions are on your plate after reading this:

1. **Confirm or amend the `davila7` adoption profile.** Default per Section 1: adopt `unreal-engine-developer` (with proactive-language tweak) and `game-designer`; defer `3d-artist`; park `unity-game-developer`; drop `game-developer`.
2. **Confirm the UnrealClaude adoption strategy.** Default per Section 2: adopt as a documented optional dependency. Don't bundle the plugin; ship the integration knowledge and let the `/onboard` command handle detection.
3. **Pick which Section 4 skills to schedule next.** The shortlist is four — UE5 anti-patterns is the obvious first pick if you want one.
4. **Decide the featureDev question.** Re-import with a constraining skill, or leave removed.

What you don't have to decide today: the install-script extension (deferred until the agent set is confirmed), the actual deploy to `~/claude/`, and the UnrealClaude clone-and-build for TBH or TOG. All of those follow from the four decisions above and happen in subsequent passes.

The work shipped this pass is the *review surface* — analysis docs, vendored files with provenance, a reference catalog, and stubs for the command, template, and README. Nothing has crossed into your active configuration. Whatever you decide, the current state is fully reversible by deleting the `GameMakerKit/` folder.